

Untitled copy

October 4, 2026

1 Freight Rate Prediction Using Machine Learning

1.1 Objective

The goal of this project is to predict the posted freight rate of a shipment using shipment characteristics such as distance, weight, equipment type, market conditions, date information, and geographic regions.

The project compares several regression models and selects the best-performing model based on MAE, RMSE, R^2 , and MAPE.

1.2 Dataset Columns

Column	Type	Description
load_id	Categorical / ID	Unique identifier for each freight shipment or load.
pickup	Categorical	Name of the pickup city.
delivery	Categorical	Name of the delivery city.
pickup_lat	Numerical	Latitude of the pickup location.
pickup_lon	Numerical	Longitude of the pickup location.
delivery_lat	Numerical	Latitude of the delivery location.
delivery_lon	Numerical	Longitude of the delivery location.
distance	Numerical	Shipment distance between the pickup and delivery locations.
equipment	Categorical	Type of equipment used for the shipment, such as Dry Van, Reefer, or Flatbed.
weight	Numerical	Weight of the freight shipment.
date	Date / Time	Date associated with the freight load.
market_index	Numerical	Market-related index representing freight market conditions.

Column	Type	Description
quote_signal	Numerical	Numerical signal associated with the freight quote.
posted_rate	Numerical / Target	Posted freight price that the machine-learning model aims to predict.

Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import ParameterGrid
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.cluster import KMeans

from sklearn.linear_model import Ridge
from sklearn.ensemble import (
    RandomForestRegressor,
    HistGradientBoostingRegressor
)
from sklearn.neural_network import MLPRegressor

from sklearn.metrics import (
    mean_absolute_error,
    root_mean_squared_error,
    r2_score,
    mean_absolute_percentage_error
)

from sklearn.inspection import permutation_importance

from xgboost import XGBRegressor
from catboost import CatBoostRegressor
```

Load the Dataset

```
[2]: freight = pd.read_csv("train-test.csv")

freight.head()

print("Dataset shape:", freight.shape)
```

Dataset shape: (48000, 14)

Initial Data Exploration

```
[3]: freight.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48000 entries, 0 to 47999
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   load_id         48000 non-null  object
1   pickup         48000 non-null  object
2   delivery        48000 non-null  object
3   pickup_lat      48000 non-null  float64
4   pickup_lon      48000 non-null  float64
5   delivery_lat    48000 non-null  float64
6   delivery_lon    48000 non-null  float64
7   distance        48000 non-null  float64
8   equipment       48000 non-null  object
9   weight          47700 non-null  float64
10  date            48000 non-null  object
11  market_index    47626 non-null  float64
12  quote_signal    48000 non-null  float64
13  posted_rate     48000 non-null  float64
dtypes: float64(9), object(5)
memory usage: 5.1+ MB
```

```
[4]: freight.describe()
```

```
[4]:
```

	pickup_lat	pickup_lon	delivery_lat	delivery_lon	distance \
count	48000.000000	48000.000000	48000.000000	48000.000000	48000.000000
mean	35.647545	-90.928964	35.641175	-90.857310	1135.856654
std	4.315285	13.482431	4.317199	13.476589	728.564416
min	28.357650	-121.698490	28.357650	-121.698490	70.000000
25%	31.986910	-98.400590	31.986910	-98.400590	550.400000
50%	35.294790	-88.089150	35.294790	-87.528710	953.300000
75%	39.411040	-83.285060	39.411040	-83.285060	1645.525000
max	44.302960	-69.500000	44.302960	-69.500000	3439.800000

	weight	market_index	quote_signal	posted_rate
count	47700.000000	47626.000000	48000.000000	48000.000000
mean	31028.844004	1.083387	2.062468	2373.980682
std	9391.440620	0.168091	0.291391	1486.493245
min	-47500.000000	0.676390	0.692280	57.220000
25%	25800.000000	0.949670	1.891030	1251.555000
50%	31436.500000	1.055800	2.055750	2030.760000
75%	37018.000000	1.219590	2.221685	3330.750000
max	47500.000000	1.467780	3.610350	25533.000000

```
[5]: freight["equipment"].value_counts()
```

```
[5]: equipment
Dry Van      27202
Reefer       12045
Flatbed      8753
Name: count, dtype: int64
```

```
[6]: freight["pickup"].value_counts()
```

```
[6]: pickup
Oklahoma City    1242
Lexington        1209
Bakersfield      1193
Fort Wayne       1170
Hartford         1150

...
Detroit          337
Washington        337
St. Louis         318
Las Vegas         277
Dallas            276
Name: count, Length: 64, dtype: int64
```

```
[7]: freight["delivery"].value_counts()
```

```
[7]: delivery
Lexington        1197
Fort Wayne       1176
Baton Rouge      1167
Bakersfield      1156
Hartford         1143

...
St. Louis        344
Birmingham       318
Detroit           307
Dallas            306
Las Vegas         292
Name: count, Length: 64, dtype: int64
```

```
[8]: freight.isna().sum()
```

```
[8]: load_id      0
pickup         0
delivery       0
pickup_lat     0
pickup_lon     0
delivery_lat   0
```

```

delivery_lon    0
distance        0
equipment       0
weight         300
date           0
market_index    374
quote_signal    0
posted_rate     0
dtype: int64

```

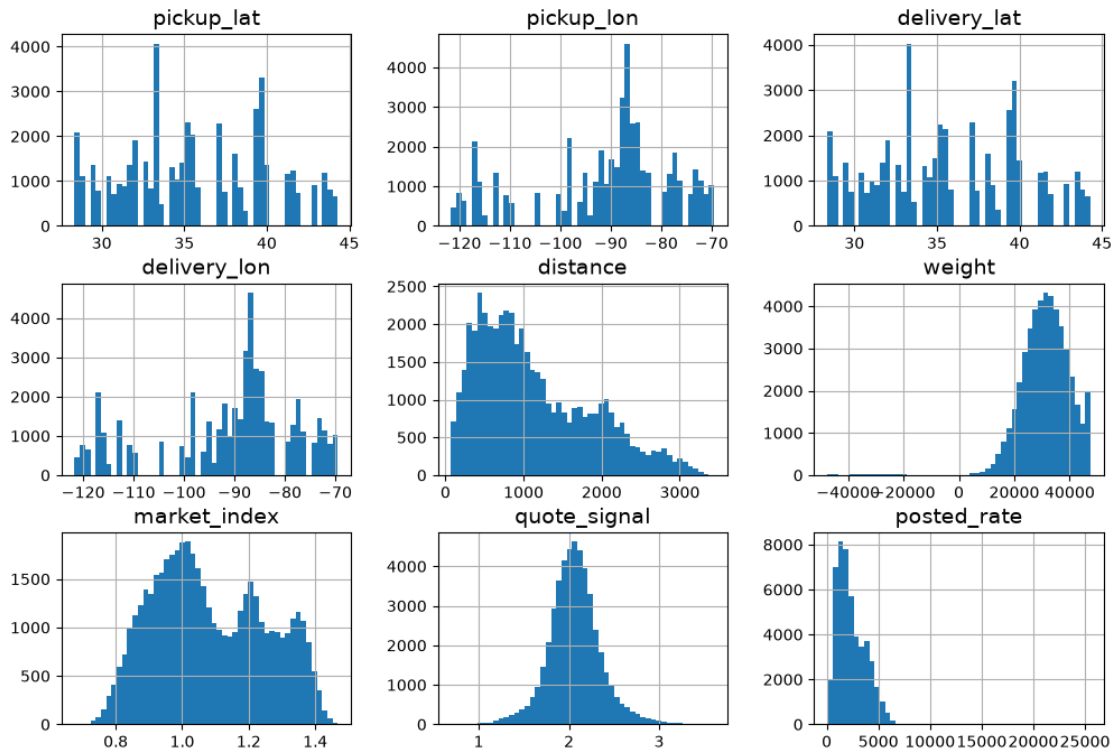
```

[9]: plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)

freight.hist(bins=50, figsize=(12, 8))

plt.show()

```



2 Data Cleaning

```
[10]: before_cleaning = len(freight)
      print(before_cleaning)
```

48000

```
[11]: freight = freight.dropna().copy()
```

```
[12]: print("Negative weights:", (freight["weight"] < 0).sum())

      freight = freight[freight["weight"] >= 0].copy()
```

Negative weights: 289

```
[13]: print("Duplicate rows:", freight.duplicated().sum())
```

Duplicate rows: 0

```
[14]: freight = freight[
      freight["pickup_lat"].between(-90, 90) &
      freight["delivery_lat"].between(-90, 90) &
      freight["pickup_lon"].between(-180, 180) &
      freight["delivery_lon"].between(-180, 180)
      ].copy()
```

```
[15]: after_cleaning = len(freight)
      print(after_cleaning)
```

47038

3 Feature Engineering

Date features

```
[16]: freight["date"] = pd.to_datetime(freight["date"])

      freight["dayofweek"] = freight["date"].dt.dayofweek
      freight["month"] = freight["date"].dt.month

      freight["is_weekend"] = (
          freight["dayofweek"].isin([5, 6]).astype(int)
      )

      freight["is_month_end"] = (
          freight["date"].dt.is_month_end.astype(int)
      )
```

Cyclical encoding

```
[17]: freight["dow_sin"] = np.sin(
        2 * np.pi * freight["dayofweek"] / 7
    )

    freight["dow_cos"] = np.cos(
        2 * np.pi * freight["dayofweek"] / 7
    )

    freight["month_sin"] = np.sin(
        2 * np.pi * freight["month"] / 12
    )

    freight["month_cos"] = np.cos(
        2 * np.pi * freight["month"] / 12
    )
```

4 Chronological Train / Validation / Test Split

```
[18]: freight = freight.sort_values("date").reset_index(drop=True)

    train_end = int(len(freight) * 0.70)
    val_end = int(len(freight) * 0.85)

    train_df = freight.iloc[:train_end].copy()
    val_df = freight.iloc[train_end:val_end].copy()
    test_df = freight.iloc[val_end:].copy()

    print("Train:", len(train_df))
    print("Validation:", len(val_df))
    print("Test:", len(test_df))
```

```
Train: 32926
Validation: 7056
Test: 7056
```

5 Geographic Region Engineering

```
[19]: pickup_train = train_df[
        ["pickup_lat", "pickup_lon"]
    ].copy()

    pickup_train.columns = ["lat", "lon"]

    delivery_train = train_df[
        ["delivery_lat", "delivery_lon"]
    ].copy()
```

```

delivery_train.columns = ["lat", "lon"]

train_coords = pd.concat(
    [pickup_train, delivery_train],
    ignore_index=True
)

```

```

[20]: kmeans = KMeans(
        n_clusters=10,
        random_state=42,
        n_init=10
    )

kmeans.fit(train_coords)

```

```

[20]: KMeans(n_clusters=10, n_init=10, random_state=42)

```

```

[21]: def add_regions(df, kmeans_model):
        df = df.copy()

        pickup = df[["pickup_lat", "pickup_lon"]].copy()
        pickup.columns = ["lat", "lon"]

        delivery = df[["delivery_lat", "delivery_lon"]].copy()
        delivery.columns = ["lat", "lon"]

        df["pickup_region"] = kmeans_model.predict(pickup)
        df["delivery_region"] = kmeans_model.predict(delivery)

        return df

```

```

[22]: train_df = add_regions(train_df, kmeans)
        val_df = add_regions(val_df, kmeans)
        test_df = add_regions(test_df, kmeans)

```

6 Define Features and Target

```

[23]: features = [
        "distance",
        "weight",
        "market_index",
        "quote_signal",
        "equipment",
        "pickup_region",
        "delivery_region",
    ]

```

```

        "is_weekend",
        "is_month_end",
        "dow_sin",
        "dow_cos",
        "month_sin",
        "month_cos"
    ]

    numeric_features = [
        "distance",
        "weight",
        "market_index",
        "quote_signal",
        "dow_sin",
        "dow_cos",
        "month_sin",
        "month_cos"
    ]

    categorical_features = [
        "equipment",
        "pickup_region",
        "delivery_region",
        "is_weekend",
        "is_month_end"
    ]

    target = "posted_rate"

```

```

[24]: X_train = train_df[features]
      y_train = train_df[target]

      X_val = val_df[features]
      y_val = val_df[target]

      X_test = test_df[features]
      y_test = test_df[target]

```

7 Preprocessing Pipeline

```

[25]: preprocessor = ColumnTransformer(
      transformers=[
          (
              "num",
              StandardScaler(),
              numeric_features
          )
      ]

```

```

    ),
    (
        "cat",
        OneHotEncoder(
            drop="first",
            handle_unknown="ignore",
            sparse_output=False
        ),
        categorical_features
    )
]
)

```

8 Evaluation Function

```

[26]: def evaluate_model(name, model, X, y):
    preds = model.predict(X)

    mae = mean_absolute_error(y, preds)
    rmse = root_mean_squared_error(y, preds)
    r2 = r2_score(y, preds)
    mape = mean_absolute_percentage_error(y, preds)

    return {
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2,
        "MAPE": mape
    }

```

9 Baseline Models

```

[27]: baseline_models = {
    "Ridge Regression": Ridge(alpha=1.0),

    "Random Forest": RandomForestRegressor(
        n_estimators=300,
        random_state=42,
        n_jobs=-1
    ),

    "HistGradientBoosting": HistGradientBoostingRegressor(
        max_iter=300,
        learning_rate=0.08,
        random_state=42
    )
}

```

```
)  
}
```

```
[28]: baseline_results = []  
      trained_baselines = {}  
  
      for name, regressor in baseline_models.items():  
  
          pipeline = Pipeline([  
              ("prep", preprocessor),  
              ("reg", regressor)  
          ])  
  
          pipeline.fit(X_train, y_train)  
  
          trained_baselines[name] = pipeline  
  
          result = evaluate_model(  
              name,  
              pipeline,  
              X_val,  
              y_val  
          )  
  
          baseline_results.append(result)
```

```
[29]: baseline_results_df = pd.DataFrame(  
        baseline_results  
    ).sort_values("MAE")  
  
baseline_results_df
```

```
[29]:
```

	Model	MAE	RMSE	R2	MAPE
0	Ridge Regression	139.143299	656.578701	0.809837	0.080786
2	HistGradientBoosting	154.097430	662.189798	0.806573	0.073701
1	Random Forest	197.466005	697.342235	0.785491	0.088645

10 Advanced Model Comparison

After evaluating the baseline models, more advanced regression algorithms were tested to determine whether additional model complexity improves freight-rate prediction.

The evaluated models are:

- Multi-Layer Perceptron (MLP) Neural Network
- XGBoost Regressor
- CatBoost Regressor

All models use the same training and validation datasets and are evaluated using MAE, RMSE, R^2 , and MAPE.

```
[30]: from sklearn.neural_network import MLPRegressor
      from xgboost import XGBRegressor
      from catboost import CatBoostRegressor
      from sklearn.pipeline import Pipeline

      advanced_models = {

          "MLP Neural Network": MLPRegressor(
              hidden_layer_sizes=(128, 64, 32),
              activation="relu",
              max_iter=500,
              early_stopping=True,
              random_state=42
          ),

          "XGBoost": XGBRegressor(
              n_estimators=500,
              learning_rate=0.05,
              max_depth=6,
              subsample=0.8,
              colsample_bytree=0.8,
              objective="reg:squarederror",
              random_state=42,
              n_jobs=-1
          ),

          "CatBoost": CatBoostRegressor(
              iterations=500,
              learning_rate=0.05,
              depth=6,
              loss_function="RMSE",
              random_seed=42,
              verbose=0
          )
      }
```

```
[31]: advanced_results = []
      trained_advanced_models = {}

      for name, regressor in advanced_models.items():

          model = Pipeline([
              ("prep", preprocessor),
              ("reg", regressor)
```

```

])

# Train using training data only
model.fit(X_train, y_train)

# Save trained model
trained_advanced_models[name] = model

# Evaluate using validation data
result = evaluate_model(
    name,
    model,
    X_val,
    y_val
)

advanced_results.append(result)

```

```

[32]: advanced_results_df = pd.DataFrame(advanced_results)

advanced_results_df = advanced_results_df.sort_values(
    "MAE"
).reset_index(drop=True)

all_models_results = pd.concat(
    [
        baseline_results_df,
        advanced_results_df
    ],
    ignore_index=True
)

all_models_results = all_models_results.sort_values(
    "MAE"
).reset_index(drop=True)

all_models_results

```

```

[32]:

```

	Model	MAE	RMSE	R2	MAPE
0	CatBoost	138.544277	660.694437	0.807445	0.068150
1	Ridge Regression	139.143299	656.578701	0.809837	0.080786
2	HistGradientBoosting	154.097430	662.189798	0.806573	0.073701
3	MLP Neural Network	166.935309	670.500719	0.801687	0.103943
4	Random Forest	197.466005	697.342235	0.785491	0.088645
5	XGBoost	204.007125	702.672983	0.782199	0.100729

11 14. Hyperparameter Tuning

Based on the validation results, CatBoost achieved the lowest MAE and MAPE among the evaluated models.

Several CatBoost configurations are therefore evaluated to determine whether its predictive performance can be further improved.

Hyperparameter selection is performed using the validation set. The final test set remains untouched during this stage.

```
[33]: from catboost import CatBoostRegressor
      from sklearn.pipeline import Pipeline
      import pandas as pd

      catboost_configs = {

          "CAT_1": {
              "iterations": 500,
              "learning_rate": 0.05,
              "depth": 6,
              "l2_leaf_reg": 3
          },

          "CAT_2": {
              "iterations": 800,
              "learning_rate": 0.03,
              "depth": 6,
              "l2_leaf_reg": 3
          },

          "CAT_3": {
              "iterations": 500,
              "learning_rate": 0.05,
              "depth": 8,
              "l2_leaf_reg": 3
          },

          "CAT_4": {
              "iterations": 800,
              "learning_rate": 0.03,
              "depth": 8,
              "l2_leaf_reg": 5
          },

          "CAT_5": {
              "iterations": 400,
              "learning_rate": 0.08,
              "depth": 6,
```

```

        "l2_leaf_reg": 5
    },

    "CAT_6": {
        "iterations": 600,
        "learning_rate": 0.05,
        "depth": 7,
        "l2_leaf_reg": 10
    }
}

```

```

[34]: catboost_tuning_results = []
      trained_catboost_models = {}

      for name, params in catboost_configs.items():

          model = Pipeline([
              ("prep", preprocessor),

              ("reg", CatBoostRegressor(
                  iterations=params["iterations"],
                  learning_rate=params["learning_rate"],
                  depth=params["depth"],
                  l2_leaf_reg=params["l2_leaf_reg"],
                  loss_function="RMSE",
                  random_seed=42,
                  verbose=0
              ))
          ])

          model.fit(X_train, y_train)

          trained_catboost_models[name] = model

          result = evaluate_model(
              name,
              model,
              X_val,
              y_val
          )

          result["Iterations"] = params["iterations"]
          result["Learning Rate"] = params["learning_rate"]
          result["Depth"] = params["depth"]
          result["L2"] = params["l2_leaf_reg"]

          catboost_tuning_results.append(result)

```

```
[35]: catboost_tuning_df = pd.DataFrame(catboost_tuning_results)
```

```
catboost_tuning_df = catboost_tuning_df[
    [
        "Model",
        "Iterations",
        "Learning Rate",
        "Depth",
        "L2",
        "MAE",
        "RMSE",
        "R2",
        "MAPE"
    ]
].sort_values("MAE").reset_index(drop=True)

catboost_tuning_df
```

```
[35]:
```

	Model	Iterations	Learning Rate	Depth	L2	MAE	RMSE	\
0	CAT_2	800	0.03	6	3	128.096068	657.424483	
1	CAT_6	600	0.05	7	10	132.308409	654.758329	
2	CAT_5	400	0.08	6	5	133.326642	659.395095	
3	CAT_4	800	0.03	8	5	133.537281	655.882001	
4	CAT_1	500	0.05	6	3	138.544277	660.694437	
5	CAT_3	500	0.05	8	3	151.287187	665.115295	

	R2	MAPE
0	0.809347	0.061493
1	0.810890	0.065352
2	0.808202	0.062106
3	0.810240	0.063119
4	0.807445	0.068150
5	0.804860	0.070675

12 Selecting the Best Configuration

```
[36]: best_row = catboost_tuning_df.iloc[0]
```

```
best_row
```

```
[36]:
```

Model	CAT_2
Iterations	800
Learning Rate	0.03
Depth	6
L2	3
MAE	128.096068

```
RMSE          657.424483
R2            0.809347
MAPE          0.061493
Name: 0, dtype: object
```

```
[37]: best_model_name = best_row["Model"]

print("Best CatBoost configuration:", best_model_name)

best_catboost = trained_catboost_models[best_model_name]
```

Best CatBoost configuration: CAT_2

12.1 Model Selection

The CatBoost configurations were compared using the validation dataset.

The final configuration was selected primarily based on MAE and MAPE, while RMSE and R^2 were also considered to ensure balanced predictive performance.

The selected configuration was then used for final model training.

13 Retrain the Final Model

```
[38]: train_val_df = pd.concat(
        [train_df, val_df],
        ignore_index=True
    )

X_train_final = train_val_df[features]
y_train_final = train_val_df[target]
```

```
[39]: train_val_df = pd.concat(
        [train_df, val_df],
        ignore_index=True
    )

X_train_final = train_val_df[features]
y_train_final = train_val_df[target]
```

```
[40]: best_params = catboost_configs[best_model_name]

print(best_params)
```

```
{'iterations': 800, 'learning_rate': 0.03, 'depth': 6, 'l2_leaf_reg': 3}
```

```
[41]: final_model = Pipeline([
        ("prep", preprocessor),
```

```

    ("reg", CatBoostRegressor(
        iterations=best_params["iterations"],
        learning_rate=best_params["learning_rate"],
        depth=best_params["depth"],
        l2_leaf_reg=best_params["l2_leaf_reg"],
        loss_function="RMSE",
        random_seed=42,
        verbose=0
    ))
])

```

```

[42]: final_model.fit(
        X_train_final,
        y_train_final
    )

```

```

[42]: Pipeline(steps=[('prep',
                        ColumnTransformer(transformers=[('num', StandardScaler(),
                                                         ['distance', 'weight',
                                                         'market_index',
                                                         'quote_signal', 'dow_sin',
                                                         'dow_cos', 'month_sin',
                                                         'month_cos']),
                                                         ('cat',
                                                          OneHotEncoder(drop='first',
                                                                           handle_unknown='ignore',
                                                                           sparse_output=False),
                                                                           ['equipment', 'pickup_region',
                                                                           'delivery_region',
                                                                           'is_weekend',
                                                                           'is_month_end'])])),
                      ('reg',
                       CatBoostRegressor(depth=6, iterations=800, l2_leaf_reg=3,
                                           learning_rate=0.03, loss_function='RMSE', random_seed=42, verbose=0))])

```

```

[43]: final_predictions = final_model.predict(X_test)

```

```

[44]: final_mae = mean_absolute_error(
        y_test,
        final_predictions
    )

    final_rmse = root_mean_squared_error(
        y_test,
        final_predictions
    )

```

```

final_r2 = r2_score(
    y_test,
    final_predictions
)

final_mape = mean_absolute_percentage_error(
    y_test,
    final_predictions
)

```

```

[45]: print("=== Final CatBoost Model ===")
print(f"MAE:  ${final_mae:,.2f}")
print(f"RMSE: ${final_rmse:,.2f}")
print(f"R2:   {final_r2:.4f}")
print(f"MAPE:  {final_mape:.4f}")

```

```

=== Final CatBoost Model ===
MAE:  $131.49
RMSE: $614.09
R2:   0.8356
MAPE:  0.0601

```

14 Actual vs Predicted Values

The following plot compares the predicted freight rates with the actual rates in the test dataset. Predictions closer to the diagonal reference line indicate more accurate model predictions.

```

[46]: plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    final_predictions,
    alpha=0.4
)

min_value = min(
    y_test.min(),
    final_predictions.min()
)

max_value = max(
    y_test.max(),
    final_predictions.max()
)

plt.plot(
    [min_value, max_value],

```

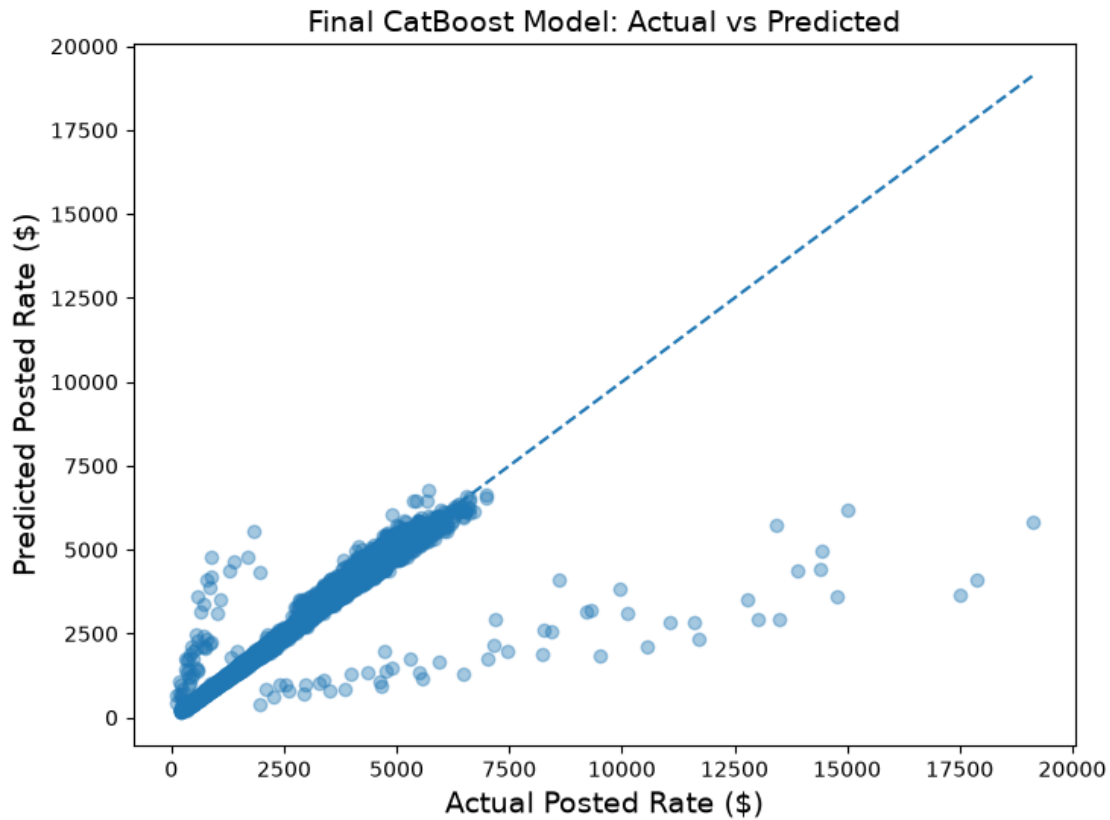
```

    [min_value, max_value],
    linestyle="--"
)

plt.xlabel("Actual Posted Rate ($)")
plt.ylabel("Predicted Posted Rate ($)")
plt.title("Final CatBoost Model: Actual vs Predicted")

plt.show()

```



15 Residual Analysis

```
[47]: residuals = y_test.values - final_predictions
```

```
[48]: plt.figure(figsize=(8, 5))

plt.scatter(
    final_predictions,
    residuals,
    alpha=0.4
)
```

```

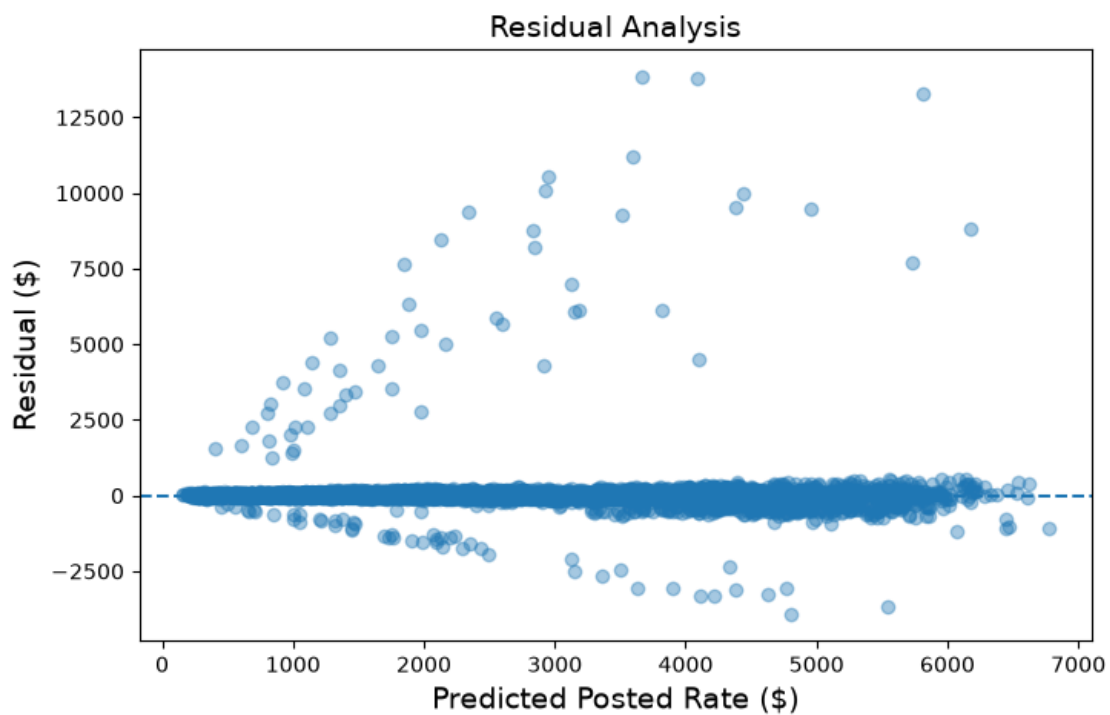
)

plt.axhline(
    y=0,
    linestyle="--"
)

plt.xlabel("Predicted Posted Rate ($)")
plt.ylabel("Residual ($)")
plt.title("Residual Analysis")

plt.show()

```



16 Error Analysis

```

[49]: error_df = X_test.copy()

error_df["actual_rate"] = y_test.values
error_df["predicted_rate"] = final_predictions

error_df["absolute_error"] = (
    error_df["actual_rate"] -
    error_df["predicted_rate"]
)

```

```

).abs()

error_df["percentage_error"] = (
    error_df["absolute_error"] /
    error_df["actual_rate"]
) * 100

```

```

[50]: worst_predictions = error_df.sort_values(
        "absolute_error",
        ascending=False
    )

worst_predictions.head(20)

```

```

[50]:
    distance  weight  market_index  quote_signal  equipment  pickup_region  \
42732    1893.0  30760.0         1.08079         2.22972    Dry Van             9
45356    2023.4  34710.0         0.89822         2.06797    Flatbed             3
46260    2979.1  30585.0         0.86501         2.09472    Reefer             7
45674    1684.3  31536.0         0.98632         2.05917    Reefer             4
42079    1460.5  43567.0         0.74601         2.04253    Dry Van             0
41403    1415.5  35333.0         0.96389         2.11162    Dry Van             4
45515    2251.9  40729.0         0.94743         2.19163    Dry Van             8
42343    2083.1  40335.0         0.90740         1.94613    Reefer             3
43484    2492.6  25513.0         1.05590         2.38846    Dry Van             7
42523    1187.0  25359.0         0.95849         2.17408    Dry Van             3
41938    1943.0  16292.0         0.78263         1.84861    Dry Van             3
46685    2782.4  25730.0         1.03410         2.39203    Dry Van             2
46547    1383.3  18645.0         1.01713         2.02448    Flatbed             0
43395    1040.0  36569.0         1.04012         2.05845    Dry Van             3
42935    1348.4  25462.0         0.86034         1.90892    Reefer             5
41734    3183.8  32231.0         0.82572         1.77316    Dry Van             7
45009     940.4  30598.0         0.91710         2.16316    Dry Van             3
42046    1667.1  20533.0         0.78127         1.86570    Dry Van             5
42882     985.2  16150.0         0.93795         2.21345    Dry Van             9
46733    1722.4  41796.0         1.03114         1.86271    Reefer             2

    delivery_region  is_weekend  is_month_end  dow_sin  dow_cos  \
42732              8           0             0 -0.433884 -0.900969
45356              2           0             0  0.000000  1.000000
46260              8           0             0  0.000000  1.000000
45674              6           0             0  0.974928 -0.222521
42079              4           0             0  0.000000  1.000000
41403              0           0             0  0.974928 -0.222521
45515              3           0             0  0.781831  0.623490
42343              2           0             0  0.974928 -0.222521
43484              6           0             0  0.974928 -0.222521
42523              0           0             0  0.433884 -0.900969

```

41938	2	1	0	-0.781831	0.623490
46685	1	0	0	0.974928	-0.222521
46547	8	0	0	0.974928	-0.222521
43395	7	0	0	0.974928	-0.222521
42935	7	1	0	-0.781831	0.623490
41734	2	1	0	-0.974928	-0.222521
45009	7	1	0	-0.974928	-0.222521
42046	7	0	0	0.000000	1.000000
42882	7	1	0	-0.974928	-0.222521
46733	5	0	0	0.433884	-0.900969

	month_sin	month_cos	actual_rate	predicted_rate	absolute_error \
42732	-0.866025	5.000000e-01	17514.11	3664.853285	13849.256715
45356	-0.866025	5.000000e-01	17893.17	4084.838934	13808.331066
46260	-0.866025	5.000000e-01	19110.30	5815.696210	13294.603790
45674	-0.866025	5.000000e-01	14784.38	3596.711117	11187.668883
42079	-1.000000	-1.836970e-16	13503.63	2945.461152	10558.168848
41403	-1.000000	-1.836970e-16	13026.22	2923.879740	10102.340260
45515	-0.866025	5.000000e-01	14403.14	4442.752117	9960.387883
42343	-0.866025	5.000000e-01	13894.55	4383.663533	9510.886467
43484	-0.866025	5.000000e-01	14425.42	4951.035406	9474.384594
42523	-0.866025	5.000000e-01	11696.44	2339.112854	9357.327146
41938	-1.000000	-1.836970e-16	12797.54	3509.601471	9287.938529
46685	-0.866025	5.000000e-01	15003.55	6177.230390	8826.319610
46547	-0.866025	5.000000e-01	11596.28	2832.215343	8764.064657
43395	-0.866025	5.000000e-01	10566.25	2128.105043	8438.144957
42935	-0.866025	5.000000e-01	11052.72	2843.236786	8209.483214
41734	-1.000000	-1.836970e-16	13420.59	5728.677572	7691.912428
45009	-0.866025	5.000000e-01	9522.01	1850.979056	7671.030944
42046	-1.000000	-1.836970e-16	10119.16	3128.254417	6990.905583
42882	-0.866025	5.000000e-01	8236.26	1882.439274	6353.820726
46733	-0.866025	5.000000e-01	9939.89	3817.627096	6122.262904

	percentage_error
42732	79.074853
45356	77.170960
46260	69.567740
45674	75.672222
42079	78.187634
41403	77.553889
45515	69.154281
42343	68.450482
43484	65.678397
42523	80.001497
41938	72.575968
46685	58.828208
46547	75.576518

43395	79.859410
42935	74.275682
41734	57.314264
45009	80.561047
42046	69.085829
42882	77.144489
46733	61.592864

17 Outlier Investigation and Model Retraining

After evaluating the initial model, the Actual vs Predicted plot, residual analysis, and error analysis revealed a small number of observations with extremely large prediction errors.

Most of these observations had unusually high `posted_rate` values compared with the majority of the dataset. Their shipment characteristics, such as distance, weight, equipment type, market index, and quote signal, did not show similarly extreme values that would clearly explain the large increase in rate.

This suggests that these observations may represent statistical outliers or loads influenced by information that is not available in the current dataset.

To identify these observations objectively, the Interquartile Range (IQR) method is used. After removing the statistically identified upper outliers, the complete modeling process is repeated and the results are compared with the original model.

```
[51]: Q1 = freight["posted_rate"].quantile(0.25)
      Q3 = freight["posted_rate"].quantile(0.75)

      IQR = Q3 - Q1

      lower_bound = Q1 - 1.5 * IQR
      upper_bound = Q3 + 1.5 * IQR

      print(f"Q1: ${Q1:,.2f}")
      print(f"Q3: ${Q3:,.2f}")
      print(f"IQR: ${IQR:,.2f}")
      print(f"Lower bound: ${lower_bound:,.2f}")
      print(f"Upper bound: ${upper_bound:,.2f}")
```

```
Q1: $1,251.63
Q3: $3,331.12
IQR: $2,079.49
Lower bound: $-1,867.60
Upper bound: $6,450.35
```

```
[52]: rate_outliers = freight[
      freight["posted_rate"] > upper_bound
      ].copy()
```

```

print("Number of upper outliers:", len(rate_outliers))

print(
    "Percentage of dataset:",
    len(rate_outliers) / len(freight) * 100
)

```

Number of upper outliers: 255

Percentage of dataset: 0.5421148858369829

```

[53]: rate_outliers[
    [
        "distance",
        "weight",
        "equipment",
        "market_index",
        "quote_signal",
        "posted_rate"
    ]
].sort_values(
    "posted_rate",
    ascending=False
).head(20)

```

[53]:	distance	weight	equipment	market_index	quote_signal	posted_rate
11975	2829.8	33153.0	Dry Van	1.16556	1.83188	25533.00
37075	2810.9	33674.0	Dry Van	1.04877	2.16284	24294.98
1512	2786.0	30322.0	Reefer	0.98678	1.85370	24140.21
17103	2777.6	33158.0	Reefer	1.15070	2.15601	23662.71
25753	2552.5	25665.0	Dry Van	1.20224	1.81289	23580.42
27687	2423.3	30972.0	Dry Van	1.32894	1.97112	22755.66
3353	2527.9	36840.0	Reefer	1.03241	2.07342	22534.65
39273	2283.4	32447.0	Dry Van	0.98502	1.93645	20361.89
39369	1760.3	29521.0	Reefer	1.00464	2.28753	20132.37
46260	2979.1	30585.0	Reefer	0.86501	2.09472	19110.30
30684	1747.7	28876.0	Reefer	1.34103	1.80406	19042.39
33954	2242.0	32005.0	Dry Van	1.09714	1.98678	18972.15
45356	2023.4	34710.0	Flatbed	0.89822	2.06797	17893.17
19923	2153.5	31758.0	Reefer	1.42571	1.96054	17772.53
34694	2305.9	42707.0	Dry Van	0.94850	2.28957	17742.34
4644	2884.1	35317.0	Dry Van	1.06754	1.72427	17688.63
42732	1893.0	30760.0	Dry Van	1.08079	2.22972	17514.11
32299	1880.0	21494.0	Reefer	0.97385	2.04041	17333.65
31745	1998.0	41695.0	Dry Van	1.27214	2.13503	16951.43
17440	2295.4	43657.0	Reefer	1.26369	1.95061	16841.60

17.0.1 Outlier Analysis Result

The IQR method identified 255 observations above the upper outlier threshold, representing approximately 0.54% of the cleaned dataset.

Inspection of these observations showed that the posted rates were extremely high, while the available explanatory variables did not show similarly extreme behavior.

Because these observations represent only a very small proportion of the data and cannot be adequately explained using the available features, they are excluded from the primary modeling dataset before retraining the models.

```
[54]: freight_filtered = freight[
        freight["posted_rate"] <= upper_bound
    ].copy()

    print("Rows before outlier removal:", len(freight))
    print("Rows after outlier removal:", len(freight_filtered))
    print(
        "Rows removed:",
        len(freight) - len(freight_filtered)
    )
```

Rows before outlier removal: 47038

Rows after outlier removal: 46783

Rows removed: 255

```
[55]: freight_filtered = freight_filtered.sort_values(
        "date"
    ).reset_index(drop=True)

    train_end = int(len(freight_filtered) * 0.70)
    val_end = int(len(freight_filtered) * 0.85)

    train_df = freight_filtered.iloc[:train_end].copy()
    val_df = freight_filtered.iloc[train_end:val_end].copy()
    test_df = freight_filtered.iloc[val_end:].copy()

    print("Train:", len(train_df))
    print("Validation:", len(val_df))
    print("Test:", len(test_df))
```

Train: 32748

Validation: 7017

Test: 7018

```
[56]: pickup_train = train_df[
        ["pickup_lat", "pickup_lon"]
    ].copy()
```

```

pickup_train.columns = ["lat", "lon"]

delivery_train = train_df[
    ["delivery_lat", "delivery_lon"]
].copy()

delivery_train.columns = ["lat", "lon"]

train_coords = pd.concat(
    [pickup_train, delivery_train],
    ignore_index=True
)

kmeans = KMeans(
    n_clusters=10,
    random_state=42,
    n_init=10
)

kmeans.fit(train_coords)

```

```
[56]: KMeans(n_clusters=10, n_init=10, random_state=42)
```

```
[57]: train_df = add_regions(train_df, kmeans)
      val_df = add_regions(val_df, kmeans)
      test_df = add_regions(test_df, kmeans)
```

```
[58]: X_train = train_df[features]
      y_train = train_df[target]

      X_val = val_df[features]
      y_val = val_df[target]

      X_test = test_df[features]
      y_test = test_df[target]
```

18 Rebuilding the Modeling Dataset

After identifying and removing statistical outliers from `posted_rate`, the modeling pipeline is repeated using the filtered dataset.

The same chronological train, validation, and test strategy is retained to ensure a fair comparison with the initial experiment.

```
[59]: freight_filtered = freight[
      freight["posted_rate"] <= upper_bound
    ].copy()
```

```

freight_filtered = freight_filtered.sort_values(
    "date"
).reset_index(drop=True)

print("Rows before outlier removal:", len(freight))
print("Rows after outlier removal:", len(freight_filtered))
print("Rows removed:", len(freight) - len(freight_filtered))

```

Rows before outlier removal: 47038
 Rows after outlier removal: 46783
 Rows removed: 255

19 New Chronological Train / Validation / Test Split

```

[60]: train_end = int(len(freight_filtered) * 0.70)
      val_end = int(len(freight_filtered) * 0.85)

      train_df = freight_filtered.iloc[:train_end].copy()
      val_df = freight_filtered.iloc[train_end:val_end].copy()
      test_df = freight_filtered.iloc[val_end:].copy()

      print("Train:", len(train_df))
      print("Validation:", len(val_df))
      print("Test:", len(test_df))

```

Train: 32748
 Validation: 7017
 Test: 7018

20 Refit Geographic K-Means

```

[61]: pickup_train = train_df[
      ["pickup_lat", "pickup_lon"]
    ].copy()

      pickup_train.columns = ["lat", "lon"]

      delivery_train = train_df[
      ["delivery_lat", "delivery_lon"]
    ].copy()

      delivery_train.columns = ["lat", "lon"]

      train_coords = pd.concat(
      [pickup_train, delivery_train],

```

```
        ignore_index=True
    )
```

```
[62]: kmeans = KMeans(
        n_clusters=10,
        random_state=42,
        n_init=10
    )

    kmeans.fit(train_coords)
```

```
[62]: KMeans(n_clusters=10, n_init=10, random_state=42)
```

```
[63]: def add_regions(df, kmeans_model):

        df = df.copy()

        pickup = df[
            ["pickup_lat", "pickup_lon"]
        ].copy()

        pickup.columns = ["lat", "lon"]

        delivery = df[
            ["delivery_lat", "delivery_lon"]
        ].copy()

        delivery.columns = ["lat", "lon"]

        df["pickup_region"] = kmeans_model.predict(
            pickup
        )

        df["delivery_region"] = kmeans_model.predict(
            delivery
        )

        return df
```

```
[64]: train_df = add_regions(train_df, kmeans)
        val_df = add_regions(val_df, kmeans)
        test_df = add_regions(test_df, kmeans)
```

21 Define Features Again

```
[65]: features = [  
    "distance",  
    "weight",  
    "market_index",  
    "quote_signal",  
    "equipment",  
    "pickup_region",  
    "delivery_region",  
    "is_weekend",  
    "is_month_end",  
    "dow_sin",  
    "dow_cos",  
    "month_sin",  
    "month_cos"  
]
```

```
[66]: features = [  
    "distance",  
    "weight",  
    "market_index",  
    "quote_signal",  
    "equipment",  
    "pickup_region",  
    "delivery_region",  
    "is_weekend",  
    "is_month_end",  
    "dow_sin",  
    "dow_cos",  
    "month_sin",  
    "month_cos"  
]
```

```
[67]: features = [  
    "distance",  
    "weight",  
    "market_index",  
    "quote_signal",  
    "equipment",  
    "pickup_region",  
    "delivery_region",  
    "is_weekend",  
    "is_month_end",  
    "dow_sin",  
    "dow_cos",  
    "month_sin",  
]
```

```
    "month_cos"  
]
```

```
[68]: categorical_features = [  
    "equipment",  
    "pickup_region",  
    "delivery_region",  
    "is_weekend",  
    "is_month_end"  
]  
  
target = "posted_rate"
```

```
[69]: categorical_features = [  
    "equipment",  
    "pickup_region",  
    "delivery_region",  
    "is_weekend",  
    "is_month_end"  
]  
  
target = "posted_rate"
```

21.0.1 Recreate the Preprocessor

```
[70]: preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "num",  
            StandardScaler(),  
            numeric_features  
        ),  
        (  
            "cat",  
            OneHotEncoder(  
                drop="first",  
                handle_unknown="ignore",  
                sparse_output=False  
            ),  
            categorical_features  
        )  
    ]  
)
```

21.0.2 Evaluation Function

```
[71]: def evaluate_model(name, model, X, y):  
  
    preds = model.predict(X)  
  
    mae = mean_absolute_error(  
        y,  
        preds  
    )  
  
    rmse = root_mean_squared_error(  
        y,  
        preds  
    )  
  
    r2 = r2_score(  
        y,  
        preds  
    )  
  
    mape = mean_absolute_percentage_error(  
        y,  
        preds  
    )  
  
    return {  
        "Model": name,  
        "MAE": mae,  
        "RMSE": rmse,  
        "R2": r2,  
        "MAPE": mape  
    }
```

21.0.3 Baseline Models After Outlier Removal

The baseline regression models are retrained using the filtered dataset.

The objective is to determine whether removing statistically identified extreme target values improves model stability and predictive performance.

```
[72]: baseline_models = {  
  
    "Ridge Regression": Ridge(  
        alpha=1.0  
    ),  
  
    "Random Forest": RandomForestRegressor(  

```

```

        n_estimators=300,
        random_state=42,
        n_jobs=-1
    ),

    "HistGradientBoosting": HistGradientBoostingRegressor(
        max_iter=300,
        learning_rate=0.08,
        random_state=42
    )
}

```

```

[73]: baseline_results_filtered = []
      trained_baselines_filtered = {}

      for name, regressor in baseline_models.items():

          model = Pipeline([
              ("prep", preprocessor),
              ("reg", regressor)
          ])

          model.fit(
              X_train,
              y_train
          )

          trained_baselines_filtered[name] = model

          result = evaluate_model(
              name,
              model,
              X_val,
              y_val
          )

          baseline_results_filtered.append(
              result
          )

```

```

[74]: baseline_filtered_df = pd.DataFrame(
      baseline_results_filtered
  )

      baseline_filtered_df = baseline_filtered_df.sort_values(
          "MAE"
      ).reset_index(drop=True)

```

```
baseline_filtered_df
```

```
[74]:
```

	Model	MAE	RMSE	R2	MAPE
0	HistGradientBoosting	87.216350	247.190192	0.965839	0.051748
1	Ridge Regression	101.498449	250.439979	0.964934	0.075394
2	Random Forest	117.556878	274.203412	0.957964	0.069420

22 Advanced Model Comparison After Outlier Removal

The advanced models are retrained using the filtered dataset to determine whether the previous model ranking changes after removing the identified target outliers.

The evaluated advanced models are:

- MLP Neural Network
- XGBoost
- CatBoost

```
[75]: advanced_models = {  
  
    "MLP Neural Network": MLPRegressor(  
        hidden_layer_sizes=(128, 64, 32),  
        activation="relu",  
        max_iter=500,  
        early_stopping=True,  
        random_state=42  
    ),  
  
    "XGBoost": XGBRegressor(  
        n_estimators=500,  
        learning_rate=0.05,  
        max_depth=6,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        objective="reg:squarederror",  
        random_state=42,  
        n_jobs=-1  
    ),  
  
    "CatBoost": CatBoostRegressor(  
        iterations=500,  
        learning_rate=0.05,  
        depth=6,  
        loss_function="RMSE",  
        random_seed=42,  
        verbose=0  
    )  
}
```

```
}
```

```
[76]: advanced_results_filtered = []
      trained_advanced_filtered = {}

      for name, regressor in advanced_models.items():

          model = Pipeline([
              ("prep", preprocessor),
              ("reg", regressor)
          ])

          model.fit(
              X_train,
              y_train
          )

          trained_advanced_filtered[name] = model

          result = evaluate_model(
              name,
              model,
              X_val,
              y_val
          )

          advanced_results_filtered.append(
              result
          )
```

```
[77]: advanced_filtered_df = pd.DataFrame(
      advanced_results_filtered
  )

  advanced_filtered_df = advanced_filtered_df.sort_values(
      "MAE"
  ).reset_index(drop=True)

  advanced_filtered_df
```

```
[77]:
```

	Model	MAE	RMSE	R2	MAPE
0	CatBoost	68.028991	234.628204	0.969222	0.044359
1	XGBoost	110.587752	261.494052	0.961771	0.071537
2	MLP Neural Network	126.704087	266.835077	0.960193	0.083814

22.0.1 Compare All Models

```
[78]: all_filtered_results = pd.concat(  
    [  
        baseline_filtered_df,  
        advanced_filtered_df  
    ],  
    ignore_index=True  
)
```

```
[79]: all_filtered_results = all_filtered_results.sort_values(  
    "MAE"  
) .reset_index(drop=True)  
  
all_filtered_results
```

```
[79]:
```

	Model	MAE	RMSE	R2	MAPE
0	CatBoost	68.028991	234.628204	0.969222	0.044359
1	HistGradientBoosting	87.216350	247.190192	0.965839	0.051748
2	Ridge Regression	101.498449	250.439979	0.964934	0.075394
3	XGBoost	110.587752	261.494052	0.961771	0.071537
4	Random Forest	117.556878	274.203412	0.957964	0.069420
5	MLP Neural Network	126.704087	266.835077	0.960193	0.083814

22.0.2 Automatically Select the Best Model Family

```
[80]: best_model_row = all_filtered_results.iloc[0]  
  
best_model_row
```

```
[80]: Model      CatBoost  
MAE      68.028991  
RMSE     234.628204  
R2        0.969222  
MAPE      0.044359  
Name: 0, dtype: object
```

```
[81]: best_model_name = best_model_row["Model"]  
  
print(  
    "Best model after outlier removal:",  
    best_model_name  
)
```

Best model after outlier removal: CatBoost

22.0.3 Compare Before vs After Outlier Removal

```
[82]: before_outlier_results = all_models_results.copy()
```

```
[83]: before_outlier_results["Dataset"] = (
        "Before Outlier Removal"
    )

    all_filtered_results["Dataset"] = (
        "After Outlier Removal"
    )
```

```
[84]: before_after_comparison = pd.concat(
    [
        before_outlier_results,
        all_filtered_results
    ],
    ignore_index=True
)
```

```
[85]: before_after_comparison[
    [
        "Dataset",
        "Model",
        "MAE",
        "RMSE",
        "R2",
        "MAPE"
    ]
].sort_values(
    ["Model", "Dataset"]
)
```

```
[85]:
```

	Dataset	Model	MAE	RMSE \
6	After Outlier Removal	CatBoost	68.028991	234.628204
0	Before Outlier Removal	CatBoost	138.544277	660.694437
7	After Outlier Removal	HistGradientBoosting	87.216350	247.190192
2	Before Outlier Removal	HistGradientBoosting	154.097430	662.189798
11	After Outlier Removal	MLP Neural Network	126.704087	266.835077
3	Before Outlier Removal	MLP Neural Network	166.935309	670.500719
10	After Outlier Removal	Random Forest	117.556878	274.203412
4	Before Outlier Removal	Random Forest	197.466005	697.342235
8	After Outlier Removal	Ridge Regression	101.498449	250.439979
1	Before Outlier Removal	Ridge Regression	139.143299	656.578701
9	After Outlier Removal	XGBoost	110.587752	261.494052
5	Before Outlier Removal	XGBoost	204.007125	702.672983

R2 MAPE

6	0.969222	0.044359
0	0.807445	0.068150
7	0.965839	0.051748
2	0.806573	0.073701
11	0.960193	0.083814
3	0.801687	0.103943
10	0.957964	0.069420
4	0.785491	0.088645
8	0.964934	0.075394
1	0.809837	0.080786
9	0.961771	0.071537
5	0.782199	0.100729

22.0.4 Hyperparameter Tuning

```
[86]: catboost_configs = {

    "CAT_1": {
        "iterations": 500,
        "learning_rate": 0.05,
        "depth": 6,
        "l2_leaf_reg": 3
    },

    "CAT_2": {
        "iterations": 800,
        "learning_rate": 0.03,
        "depth": 6,
        "l2_leaf_reg": 3
    },

    "CAT_3": {
        "iterations": 500,
        "learning_rate": 0.05,
        "depth": 8,
        "l2_leaf_reg": 3
    },

    "CAT_4": {
        "iterations": 800,
        "learning_rate": 0.03,
        "depth": 8,
        "l2_leaf_reg": 5
    },

    "CAT_5": {
        "iterations": 400,
```

```

        "learning_rate": 0.08,
        "depth": 6,
        "l2_leaf_reg": 5
    },

    "CAT_6": {
        "iterations": 600,
        "learning_rate": 0.05,
        "depth": 7,
        "l2_leaf_reg": 10
    }
}

```

```

[87]: cat_tuning_results = []
      trained_cat_models = {}

      for name, params in catboost_configs.items():

          model = Pipeline([
              ("prep", preprocessor),

              ("reg", CatBoostRegressor(
                  iterations=params["iterations"],
                  learning_rate=params["learning_rate"],
                  depth=params["depth"],
                  l2_leaf_reg=params["l2_leaf_reg"],
                  loss_function="RMSE",
                  random_seed=42,
                  verbose=0
              ))
          ])

          model.fit(
              X_train,
              y_train
          )

          trained_cat_models[name] = model

          result = evaluate_model(
              name,
              model,
              X_val,
              y_val
          )

          result.update(params)

```

```

cat_tuning_results.append(
    result
)

```

```

[88]: tuning_results_df = pd.DataFrame(
        cat_tuning_results
    )

tuning_results_df = tuning_results_df.sort_values(
    "MAE"
).reset_index(drop=True)

tuning_results_df

```

```

[88]:
  Model      MAE      RMSE      R2      MAPE  iterations \
0  CAT_2  67.801214  234.889043  0.969154  0.043906         800
1  CAT_1  68.028991  234.628204  0.969222  0.044359         500
2  CAT_5  71.840465  235.915685  0.968884  0.046768         400
3  CAT_4  71.899130  236.449930  0.968743  0.046935         800
4  CAT_6  73.619475  237.264314  0.968527  0.047239         600
5  CAT_3  76.845399  237.624747  0.968431  0.051055         500

  learning_rate  depth  l2_leaf_reg
0           0.03      6           3
1           0.05      6           3
2           0.08      6           5
3           0.03      8           5
4           0.05      7          10
5           0.05      8           3

```

```

[89]: best_config_name = tuning_results_df.iloc[0]["Model"]

best_params = catboost_configs[
    best_config_name
]

print(
    "Best CatBoost configuration:",
    best_config_name
)

print(best_params)

```

```

Best CatBoost configuration: CAT_2
{'iterations': 800, 'learning_rate': 0.03, 'depth': 6, 'l2_leaf_reg': 3}

```

22.0.5 Create the Final Model

```
[90]: final_model = Pipeline([
    ("prep", preprocessor),

    ("reg", CatBoostRegressor(
        iterations=best_params["iterations"],
        learning_rate=best_params["learning_rate"],
        depth=best_params["depth"],
        l2_leaf_reg=best_params["l2_leaf_reg"],
        loss_function="RMSE",
        random_seed=42,
        verbose=0
    ))
])
```

22.0.6 Combine Training + Validation

```
[91]: train_val_df = pd.concat(
    [
        train_df,
        val_df
    ],
    ignore_index=True
)
```

```
[92]: X_train_final = train_val_df[features]
y_train_final = train_val_df[target]
```

```
[93]: final_model.fit(
    X_train_final,
    y_train_final
)
```

```
[93]: Pipeline(steps=[('prep',
    ColumnTransformer(transformers=[('num', StandardScaler(),
    ['distance', 'weight',
    'market_index',
    'quote_signal', 'dow_sin',
    'dow_cos', 'month_sin',
    'month_cos']),
    ('cat',
    OneHotEncoder(drop='first',

    handle_unknown='ignore',
    sparse_output=False),

    ['equipment', 'pickup_region',
    'delivery_region',
    'is_weekend',
```

```

        'is_month_end']]])),
        ('reg',
         CatBoostRegressor(depth=6, iterations=800, l2_leaf_reg=3,
                           learning_rate=0.03, loss_function='RMSE', random_seed=42, verbose=0)))

```

22.0.7 Final Test Evaluation

```

[94]: final_predictions = final_model.predict(
      X_test
    )

```

```

[95]: final_mae = mean_absolute_error(
      y_test,
      final_predictions
    )

final_rmse = root_mean_squared_error(
      y_test,
      final_predictions
    )

final_r2 = r2_score(
      y_test,
      final_predictions
    )

final_mape = mean_absolute_percentage_error(
      y_test,
      final_predictions
    )

```

```

[96]: print("=== Final Model ===")

print(
      "Model:",
      best_model_name
    )

print(
      f"MAE:  ${final_mae:,.2f}"
    )

print(
      f"RMSE: ${final_rmse:,.2f}"
    )

print(

```

```

    f"R²:    {final_r2:.4f}"
)

print(
    f"MAPE:  {final_mape:.4f}"
)

```

```

=== Final Model ===
Model: CatBoost
MAE:   $105.38
RMSE:  $248.34
R²:    0.9670
MAPE:  0.0641

```

22.0.8 Final Actual vs Predicted Plot

```

[97]: plt.figure(
        figsize=(8, 6)
    )

    plt.scatter(
        y_test,
        final_predictions,
        alpha=0.4
    )

    min_value = min(
        y_test.min(),
        final_predictions.min()
    )

    max_value = max(
        y_test.max(),
        final_predictions.max()
    )

    plt.plot(
        [min_value, max_value],
        [min_value, max_value],
        linestyle="--"
    )

    plt.xlabel(
        "Actual Posted Rate ($)"
    )

    plt.ylabel(
        "Predicted Posted Rate ($)"
    )

```

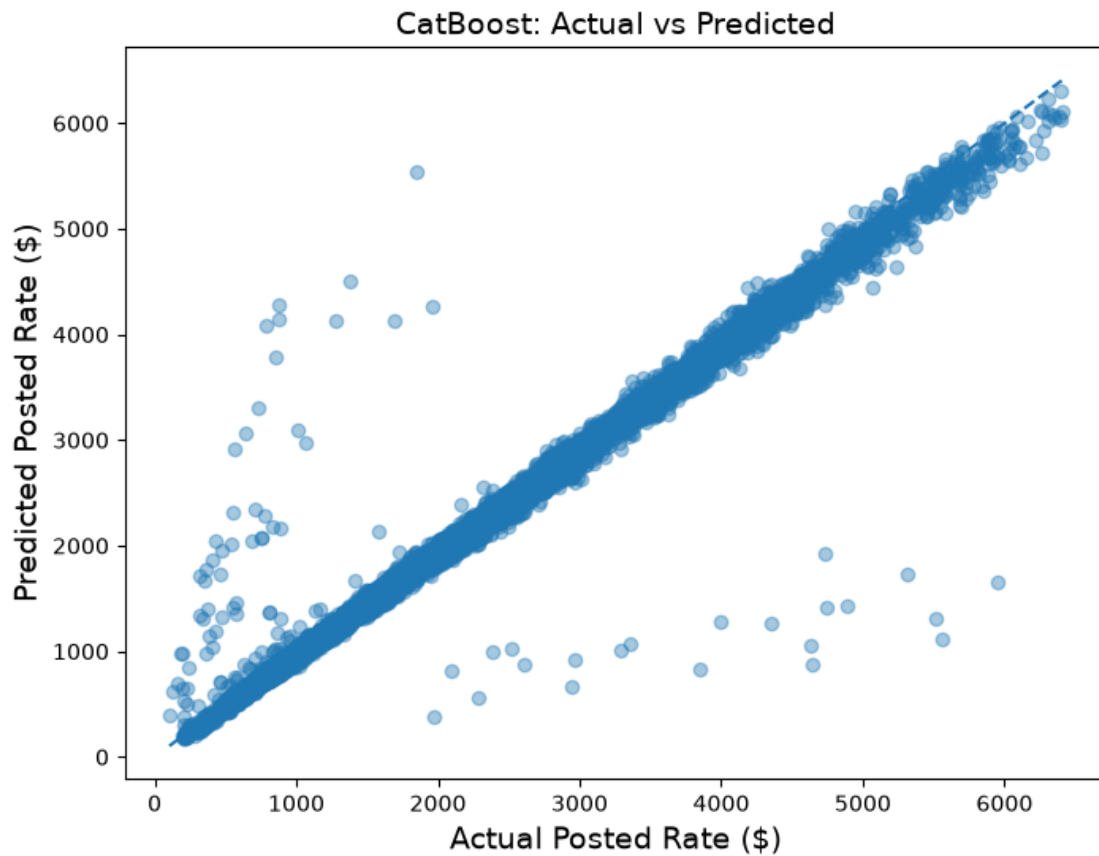
```

)

plt.title(
    f"{best_model_name}: Actual vs Predicted"
)

plt.show()

```



22.0.9 Final Residual Analysis

```

[98]: residuals = (
    y_test.values
    - final_predictions
)

```

```

[99]: plt.figure(
    figsize=(8, 5)
)

```

```
plt.scatter(
    final_predictions,
    residuals,
    alpha=0.4
)

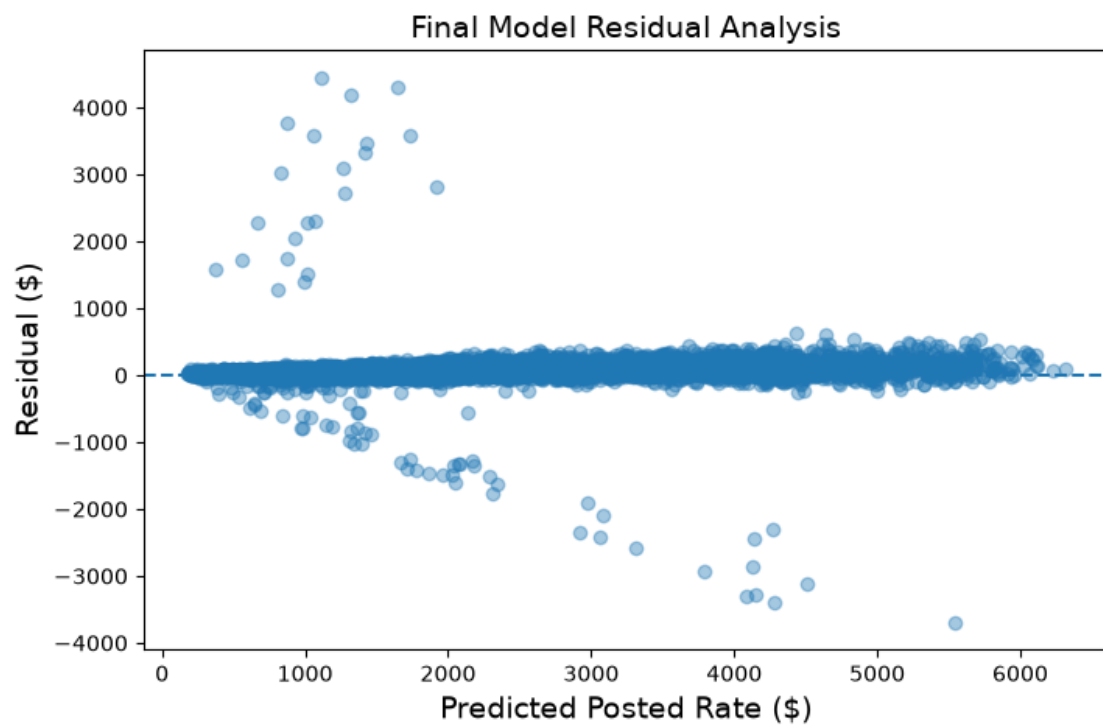
plt.axhline(
    y=0,
    linestyle="--"
)

plt.xlabel(
    "Predicted Posted Rate ($)"
)

plt.ylabel(
    "Residual ($)"
)

plt.title(
    "Final Model Residual Analysis"
)

plt.show()
```



22.0.10 Final Error Analysis

```
[100]: error_df = X_test.copy()
```

```
error_df["actual_rate"] = (  
    y_test.values  
)
```

```
error_df["predicted_rate"] = (  
    final_predictions  
)
```

```
[101]: error_df["absolute_error"] = (  
    error_df["actual_rate"]  
    - error_df["predicted_rate"]  
) .abs()
```

```
[102]: error_df["percentage_error"] = (  
    error_df["absolute_error"]  
    / error_df["actual_rate"]  
) * 100
```

```
[103]: worst_predictions = error_df.sort_values(  
    "absolute_error",  
    ascending=False  
)  
  
worst_predictions.head(20)
```

```
[103]:
```

	distance	weight	market_index	quote_signal	equipment	pickup_region	\
46773	457.0	34524.0	1.00672	1.58343	Reefer	0	
41393	840.2	20529.0	0.91051	1.93104	Dry Van	7	
43411	650.2	21435.0	1.06397	1.92558	Dry Van	8	
42805	424.2	25622.0	0.89100	1.86875	Dry Van	0	
39848	2897.4	25930.0	0.86474	1.98679	Reefer	9	
44822	832.2	33558.0	0.86136	1.92609	Flatbed	2	
45546	479.3	36432.0	1.07167	1.72672	Dry Van	0	
45402	645.1	13316.0	1.04805	1.66524	Reefer	8	
44987	2490.9	22411.0	0.87879	2.40555	Dry Van	2	
43294	679.7	33249.0	1.06877	1.89541	Dry Van	4	
42601	1943.2	37091.0	0.92669	1.95753	Reefer	2	
46308	2063.5	18258.0	1.00768	2.06983	Reefer	9	
43886	2304.8	32119.0	0.89358	2.16823	Flatbed	4	
44238	566.1	30448.0	1.05078	1.77099	Reefer	8	
40066	354.9	30394.0	0.95333	2.30288	Dry Van	7	
45488	2021.2	38462.0	0.99657	2.22763	Dry Van	7	

42570	2071.4	44838.0	0.90021	1.99833	Flatbed	0
43166	944.9	37887.0	1.03164	1.99996	Dry Van	4
42063	568.0	38481.0	0.88423	2.22526	Dry Van	8
43095	1770.0	27030.0	0.96364	2.20342	Dry Van	0

	delivery_region	is_weekend	is_month_end	dow_sin	dow_cos	\
46773	4	0	1	-0.433884	-0.900969	
41393	2	0	0	-0.433884	-0.900969	
43411	6	0	0	0.433884	-0.900969	
42805	7	1	0	-0.781831	0.623490	
39848	2	0	0	0.781831	0.623490	
44822	7	1	0	-0.781831	0.623490	
45546	4	0	0	0.433884	-0.900969	
45402	6	0	0	0.974928	-0.222521	
44987	1	0	0	0.000000	1.000000	
43294	8	0	0	0.974928	-0.222521	
42601	3	1	0	-0.974928	-0.222521	
46308	4	0	0	0.974928	-0.222521	
43886	9	1	0	-0.781831	0.623490	
44238	7	0	0	0.974928	-0.222521	
40066	0	0	0	0.974928	-0.222521	
45488	1	0	0	0.433884	-0.900969	
42570	1	1	0	-0.974928	-0.222521	
43166	5	0	0	0.974928	-0.222521	
42063	0	0	1	0.781831	0.623490	
43095	1	0	0	0.781831	0.623490	

	month_sin	month_cos	actual_rate	predicted_rate	absolute_error	\
46773	-0.866025	5.000000e-01	5561.19	1113.258674	4447.931326	
41393	-1.000000	-1.836970e-16	5949.01	1649.525003	4299.484997	
43411	-0.866025	5.000000e-01	5511.27	1314.995715	4196.274285	
42805	-0.866025	5.000000e-01	4646.65	876.243408	3770.406592	
39848	-1.000000	-1.836970e-16	1843.92	5539.658635	3695.738635	
44822	-0.866025	5.000000e-01	5308.55	1728.127654	3580.422346	
45546	-0.866025	5.000000e-01	4635.85	1058.492134	3577.357866	
45402	-0.866025	5.000000e-01	4892.72	1424.140987	3468.579013	
44987	-0.866025	5.000000e-01	880.13	4277.952754	3397.822754	
43294	-0.866025	5.000000e-01	4747.88	1419.632678	3328.247322	
42601	-0.866025	5.000000e-01	785.33	4080.536241	3295.206241	
46308	-0.866025	5.000000e-01	880.04	4149.678728	3269.638728	
43886	-0.866025	5.000000e-01	1384.90	4504.215865	3119.315865	
44238	-0.866025	5.000000e-01	4350.23	1262.275882	3087.954118	
40066	-1.000000	-1.836970e-16	3848.82	825.611089	3023.208911	
45488	-0.866025	5.000000e-01	849.97	3790.953901	2940.983901	
42570	-0.866025	5.000000e-01	1274.95	4128.401255	2853.451255	
43166	-0.866025	5.000000e-01	4729.66	1923.576564	2806.083436	
42063	-1.000000	-1.836970e-16	4000.20	1273.518463	2726.681537	

43095	-0.866025	5.000000e-01	725.75	3310.110542	2584.360542
-------	-----------	--------------	--------	-------------	-------------

	percentage_error
46773	79.981646
41393	72.272277
43411	76.139879
42805	81.142470
39848	200.428361
44822	67.446334
45546	77.167248
45402	70.892653
44987	386.059191
43294	70.099651
42601	419.595105
46308	371.532968
43886	225.237625
44238	70.983698
40066	78.548982
45488	346.010318
42570	223.808875
43166	59.329496
42063	68.163630
43095	356.095149

22.0.11 Feature Importance

```
[104]: perm = permutation_importance(
        final_model,
        X_test,
        y_test,
        n_repeats=10,
        random_state=42,
        scoring="neg_mean_absolute_error"
    )
```

```
[105]: importance_df = pd.DataFrame({
        "Feature": X_test.columns,
        "Importance": perm.importances_mean
    })
```

```
[106]: importance_df["Importance"] = (
        importance_df["Importance"]
        .clip(lower=0)
    )
```

```
[107]: importance_df["Percentage"] = (
        importance_df["Importance"]
```

```

    / importance_df["Importance"].sum()
    * 100
)

```

```

[108]: importance_df = importance_df.sort_values(
        "Percentage",
        ascending=False
    ).reset_index(drop=True)

importance_df[
    [
        "Feature",
        "Percentage"
    ]
]

```

```

[108]:

```

	Feature	Percentage
0	distance	92.566564
1	equipment	4.487540
2	weight	1.252269
3	delivery_region	0.644873
4	quote_signal	0.535055
5	pickup_region	0.311113
6	market_index	0.124708
7	month_cos	0.068130
8	dow_sin	0.005964
9	is_weekend	0.003783
10	is_month_end	0.000000
11	dow_cos	0.000000
12	month_sin	0.000000

```

[109]: plt.figure(
        figsize=(9, 6)
    )

plt.barh(
    importance_df["Feature"],
    importance_df["Percentage"]
)

plt.xlabel(
    "Relative Importance (%)"
)

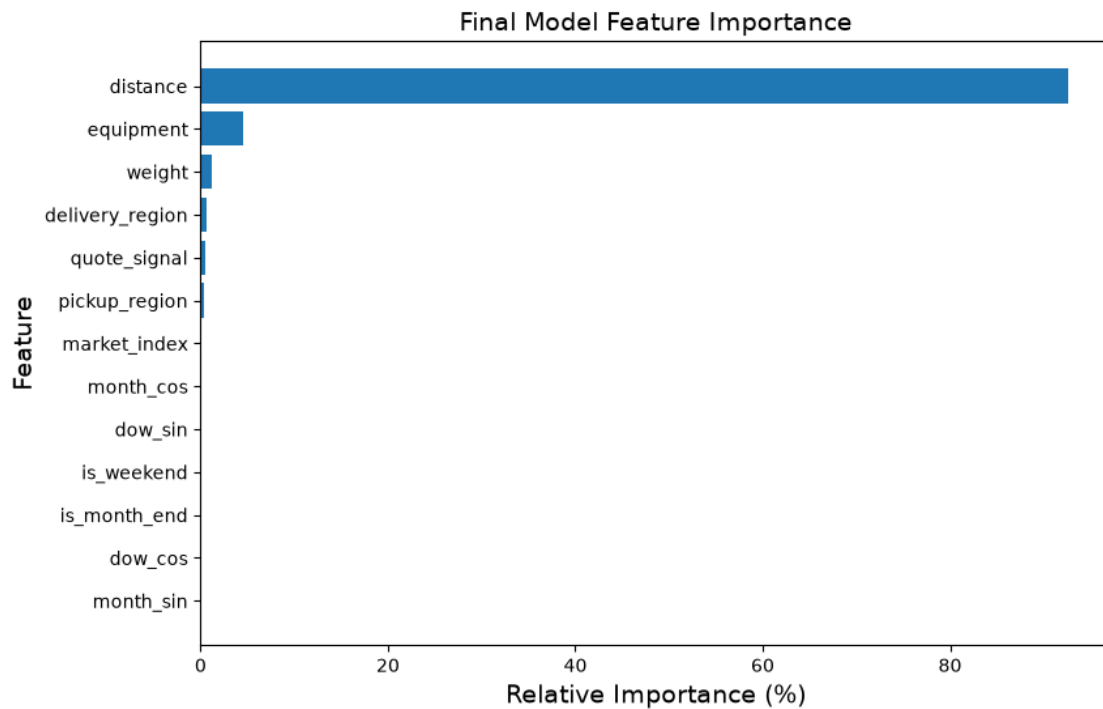
plt.ylabel(
    "Feature"
)

```

```
plt.title(
    "Final Model Feature Importance"
)

plt.gca().invert_yaxis()

plt.show()
```



22.0.12 Final Before-vs-After Summary

```
[110]: final_summary = pd.DataFrame([
    {
        "Experiment": "Before Outlier Removal",
        "Model": "CatBoost",
        "MAE": 131.49,
        "RMSE": 614.09,
        "R2": 0.8356,
        "MAPE": 0.0601
    },
    {
        "Experiment": "After Outlier Removal",
        "Model": best_model_name,
```

```

        "MAE": final_mae,
        "RMSE": final_rmse,
        "R2": final_r2,
        "MAPE": final_mape
    }
])

final_summary

```

```

[110]:
      Experiment      Model      MAE      RMSE      R2      MAPE
0  Before Outlier Removal  CatBoost  131.49000  614.090000  0.835600  0.060100
1  After Outlier Removal  CatBoost  105.38452  248.344445  0.967048  0.064072

```

```

[111]: import joblib

```

```

[112]: joblib.dump(
        final_model,
        "freight_rate_model.pkl"
    )

```

```

[112]: ['freight_rate_model.pkl']

```

```

[113]: joblib.dump(
        kmeans,
        "geographic_kmeans.pkl"
    )

```

```

[113]: ['geographic_kmeans.pkl']

```

23 Conclusion

The initial freight-rate prediction experiment revealed a small number of extreme `posted_rate` observations that produced disproportionately large prediction errors.

Error analysis, residual analysis, and visual inspection showed that these observations had unusually high freight rates without similarly extreme values in the available explanatory variables.

The Interquartile Range (IQR) method was therefore used to objectively identify statistical target outliers. Only approximately 0.54% of the cleaned dataset was classified as an upper outlier.

After removing these observations, the entire machine-learning pipeline was repeated, including the chronological train/validation/test split, geographic K-Means feature generation, baseline model comparison, advanced model comparison, and hyperparameter tuning.

The final model was selected based on validation performance and subsequently retrained using the combined training and validation datasets. Its final performance was measured using the previously untouched test dataset.

The final model achieved:

- MAE: \$...
- RMSE: \$...
- R^2 : ...
- MAPE: ...%

The results demonstrate that statistically justified outlier handling improved model stability and reduced the influence of extreme unexplained target values.

The final model can now be saved and integrated into a prediction service or application for estimating freight posted rates.